

KAFKA

DATABASE VS KAFKA

Throughput	Latency
Amount of work completed per unit	Time taken for one operation to
Quantity	Speed
How much?	How fast?
Maximize processing volume	Minimize response time
Customers served per hour	Waiting time for one customer
Primarily optimized for high throughput	Good latency, but throughput is priority

WHY DOES A DATABASE SLOW DOWN UNDER HEAVY LOAD?

Database does a lot of work including Finding, Locking, Updating, etc for every operation. Database pages scattered. So they heavily rely on random access I/O. They also guarantee ACID Properties which is expensive and time consuming to maintain.

WHY IS KAFKA FAST EVEN THOUGH IT WRITES TO DISK?

Kafka's workload is simpler. Kafka achieves high throughput through append-only logs and sequential disk I/O, avoiding expensive random disk operations.

OS Page Cache : Kafka doesn't read disk every time. OS stores recently accessed data in memory. Most reads come from here not disk.

WHY WE CANT USE KAFKA AS A DATABASE

Database	Kafka
Long-term structured data storage	Real-time event streaming
Indexes for fast lookups	Sequential event logs
Transactions (ACID)	High-throughput event processing
Constraints & data validation	Buffering and decoupling
Complex queries (SQL)	Event replay and consumption
Updates & Deletes	Append-only writes

COMMUNICATION MODELS

POLLING

Traditional systems use `request-response` communication, where services repeatedly query databases or APIs to check for new data.

LIMITATIONS:

- **Tight coupling** between services
- Increased database load due to frequent queries
- Higher latency as changes are detected only during polling intervals
- Limited scalability as the number of consumers grows

EVENT STREAMING

Event Streaming is the continuous capture, storage, processing, and delivery of events generated by various sources in real time. Event streaming platforms like Kafka enable producers to publish events and consumers to `process them asynchronously` through a distributed, append only commit log. Event streaming introduces an `event-driven architecture` where producers publish events once and multiple consumers process them `independently`.

ADVANTAGE	DISADVANTAGE
Decouples producers : consumers	Architectural complexity
Asynchronous communication	Careful partitioning strategy
Supports real-time event processing	Ordering guaranteed only within a partition
Allows event replay and auditing	Debugging distributed event flows can be difficult
Supports high-throughput	Higher operational overhead
Enables horizontal scalability through partitions	Additional infrastructure requirements
Fault tolerance via replication	Data consistency management can be challenging

APACHE KAFKA

WHAT AND WHY

Apache Kafka is a distributed event-streaming platform that enables applications to publish, store, process, and consume streams of events in a scalable, fault-tolerant, and real-time manner.

Kafka acts as an intermediary layer between event producers and event consumers, allowing them to communicate asynchronously through persistent event logs.

EVENT DRIVEN ARCHITECTURE

Event-Driven Architecture is a software architecture pattern in which system components communicate by **producing and consuming events rather than invoking each other directly**. Kafka acts as the central event broker that decouples producers from consumers and enables asynchronous event-driven communication.

QUEUE MODEL	PUB SUB MODEL
Each message is consumed by exactly one consumer.	Each message is delivered to multiple subscribers.
Work Distribution Load Balancing	Event Distribution Broadcasting
Task Queues, Order Processing, Job Scheduling	Notifications, Analytics, Monitoring, Event-Driven Systems
Consumers belong to the same Consumer Group	Consumers belong to different Consumer Groups
Each partition assigned to one consumer within the group	Each consumer group receives the same event independently

KAFKA SUPPORTS BOTH MODEL

If multiple consumers belong to the same consumer group, Kafka divides the messages among them so that each message is processed by only one consumer. This provides Queue behavior.

If consumers belong to different consumer groups, Kafka allows each group to read the same messages independently. This provides Pub/Sub behavior.

ERROR HANDLING

RETRY MECHANISM

The consumer pauses and retries the specific message a set number of times. If it continually fails, it is usually sent to a DLQ.

Main Topic → Consumer → Failure → Retry Topic → Still Fails → DLQ

DEAD LETTER QUEUE (DLQ)

The failed message is moved to a separate "error topic" so your main pipeline can keep processing valid messages without blocking. You can later analyze or fix the message.

SCHEMA REGISTRY

A Kafka Schema Registry is an external service that resides outside the Kafka cluster to manage and version data schemas for producers and consumers.

Because Apache Kafka inherently is **schema less**, treats all messages as raw byte arrays without verifying their internal structures, a schema registry serves as the central data contract to ensure structured data is safely exchanged without breaking downstream applications.

LAG MONITORING

Consumer Lag is the difference between the latest offset available in a partition and the offset last processed by a consumer. Consumer Lag indicates whether consumers are processing messages as fast as producers are generating them.

It is the primary metric used to determine the health and performance of a Kafka consumer application.

CAUSES :

SLOW CONSUMER LOGIC / DATABASE BOTTLENECK / EXTERNAL API DELAYS

SOLUTIONS :

ADD MORE CONSUMER / OPTIMIZE CONSUMER LOGIC / SCALE DATABASES/APIs

COMPACTION

Key Based cleanup which deletes older values of same key.

TERMINOLOGY

EVENT

An immutable record of a fact recording that "something happened" in the system. It is the fundamental unit of data in Apache Kafka.

FOUR MAIN COMPONENTS :

- KEY: IDENTIFIES THE EVENT (E.G. A USER ID).
- VALUE: THE ACTUAL DATA OR PAYLOAD OF WHAT HAPPENED.
- TIMESTAMP: THE EXACT TIME THE EVENT OCCURRED.
- METADATA: OPTIONAL HEADERS LIKE VERSIONING OR TRACING.

TOPIC

A Topic is a logical category or stream within Kafka used to organize related events. Producers publish events to topics, and consumers subscribe to topics to consume those events.

A Topic is a logical abstraction. [Actual storage happens in partitions.](#) Every topic contains at least one partition.

NECESSITY :

- LOGICAL CATEGORY USED TO ORGANIZE RELATED EVENTS.
- MESSAGES REMAIN EVEN AFTER CONSUMPTION. UNLIKE QUEUES.
- INDEPENDENT RETENTION POLICIES.
- TO SEPARATE EVENT STREAMS
- ENABLE INDEPENDENT PRODUCERS AND CONSUMERS.

PARTITION

Physical subdivision of a Kafka Topic where events are actually stored. Each partition is an ordered, immutable sequence of events identified by offsets. A Topic is logical concept, while Partition is the actual storage unit. [One partition → One Consumer!](#) within a customer group. If partitions exceed consumers, a consumer may handle multiple partitions. But not vice versa, this would lead to idle customer.

NECESSITY :

- PARTITIONS SPLITS INCOMING DATA
- ENABLE HORIZONTAL SCALABILITY
- ENABLE PARALLEL PROCESSING
- MAINTAIN EVENT ORDERING
- DISTRIBUTE LOAD ACROSS BROKERS

ORDERING GUARANTEE

Kafka guarantees ordering only **within** a partition.

Because events are appended sequentially inside a partition, as multiple partitions operate independently.

Order Created → Order Paid → Order Delivered (Order Preserved)

HASH KEY

A Hash Key is the event key used by Kafka to determine the partition in which an event will be stored. Hash keys ensure events with the **same key are routed to the same partition**, preserving ordering. If no key is provided, Kafka typically uses round-robin partitioning and distributes events across partitions. Better load distribution.

OFFSET

A Kafka offset is a sequential, incremental integer assigned to every message as it is appended to a partition. **"source of truth"**

RESPONSIBILITIES

- It acts as an **immutable, unique ID** for messages within a specific partition, allowing Kafka to maintain strict ordering.
- Consumer progress, recovery, replay, and fault tolerance are all determined by offsets.
- This ensures that if a consumer crashes or is restarted, it can seamlessly resume processing exactly where it left off.

REPLICATION

Replication is the process of maintaining multiple copies of a partition across different brokers to ensure fault tolerance, high availability, and data durability. Kafka performs **replication at the partition level**, not at the topic level. If one broker fails, another replica can continue serving requests.

REPLICATION FACTOR

Number of copies maintained for each partition.

Total Replicas = Partitions × Replication Factor

TOO MANY PARTITIONS ANTI-PATTERN

EACH PARTITION REQUIRES: METADATA / MEMORY / FILE HANDLES /
REPLICATION TRACKING / LEADER ELECTION TRACKING
partitions improve parallelism but have high overhead.

KAFKA Cluster

BROKER 1

TOPIC A

PARTITION 0

PARTITION 1

TOPIC B

PARTITION 0

PARTITION 1

BROKER 2

TOPIC A

PARTITION 0

PARTITION 1

TOPIC D

PARTITION 0

PARTITION 1

BROKER 3

TOPIC E

PARTITION 0

PARTITION 1

TOPIC A

PARTITION 0

PARTITION 1

LEADER ELECTION WITH REPLICATION FACTOR 3

STEP 1 : CHOOSE LEADERS

Kafka assigns one leader per partition.

PARTITION 0 → LEADER : BROKER 1
PARTITION 1 → LEADER : BROKER 2

STEP 2 : PRODUCER WRITES

Suppose hash sends it to PARTITION 0, Then producer talks ONLY to it's assigned leader. Which is BROKER 1. Producer never writes directly to followers.

STEP 3 : REPLICATION

Then followers copy from leader. Now all the 3 brokers have same data with offset of let's say 100.

STEP 4 : IN-SYNC REPLICA

Broker 2 = Offset 100

Broker 3 = Offset 100

therefore ISR = {Broker2, Broker3}

STEP 5 : LEADER CRASH

Now Kafka must choose a new leader. Kafka chooses one of the ISR replicas according to its controller's leader-election logic.

PRODUCER

A client application that publishes events or messages to a Kafka cluster.

RESPONSIBILITY :

- Create events
- Send events to Kafka
- Choose partition (directly or through key hashing)
- Receive acknowledgment from Kafka

CONSUMER

a client application that reads streams of events or messages from Apache Kafka topics. Consumers read data by pulling messages from Kafka brokers at their own pace, processing them, and then committing their read progress.

RESPONSIBILITY :

- Subscribes to topics.
- Reads events sequentially.
- Tracks offsets.
- Commits consumed offsets.

CONSUMER GROUP

A collection of independent consumer instances that cooperate to consume data from a topic. By assigning each topic partition to exactly one consumer in the group, Kafka ensures horizontal scalability and fault tolerance without duplicate message processing.

RESPONSIBILITY :

- Horizontal Scaling: Kafka assigns partitions exclusively to a single consumer in a group. To maximize parallel processing, the number of active consumers in a group should ideally equal the number of partitions in the topic.
- Offset Tracking: Groups maintain their own progression offsets for each partition. If a consumer fails or restarts, the group coordinator tracks where the group left off, allowing processing to safely resume.
- Independent Broadcasts: Multiple consumer groups reading from the exact same topic receive their own copy of the messages, which enables completely different applications to use the same data.
- Rebalancing: When consumers join or leave a group, Kafka triggers a "rebalance" to automatically redistribute the partitions among the active members.

LEADER REPLICA

The Leader Replica is the replica responsible for handling all reads and writes for a partition. Every partition has exactly one leader at any given time.

RESPONSIBILITY :

- Accept producer writes
- Serve consumer reads
- Coordinate replication
- Maintain partition state

FOLLOWER REPLICA

A Follower Replica is a replica that continuously copies data from the leader replica. Followers do not handle client reads or writes.

RESPONSIBILITY :

- Replicate leader data
- Maintain backup copies
- Stay synchronized
- Participate in failover

IN SYNC REPLICAS

ISR is the set of follower replicas that are fully synchronized with the leader and eligible for leader election.

- Producer creates an event.
- Producer sends the event to a Kafka topic.
- Kafka selects a partition for the event based on the key or partitioning strategy.
- The event is appended to the end of the selected partition.
- The broker assigns an offset to the event within that partition.
- The broker stores the event durably on disk.
- The leader partition replicates the event to follower replicas.
- Consumer groups subscribe to the topic.
- Kafka assigns partitions to consumers within each group.
- Consumers read events process event according to app logic.
- Consumers commit offsets after successful processing.
- Kafka records the committed offsets for each consumer group.
- The event remains stored in Kafka according to the configured retention policy.
- Existing or new consumer groups can replay the event by reading from earlier offsets.
- Kafka eventually removes the event when the retention period expires.

DELIVERY SEMANTICS

Producer ---> Kafka ---> Consumer

What if something crashes?

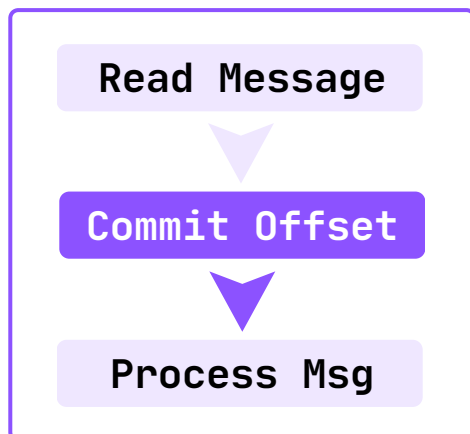
- Producer crashes
- Broker crashes
- Consumer crashes
- Network dies
- Acknowledgement gets lost

It defines the reliability guarantees for data movement between producers, brokers, and consumers. The framework provides three standard delivery models to balance trade-offs between throughput, latency, and fault tolerance.

1. AT MOST ONCE (ACKS=0)

"BETTER TO LOSE A MESSAGE THAN PROCESS IT TWICE."

Messages are delivered a maximum of one time. If a failure or network timeout occurs, Kafka does not retry, meaning messages can be lost but are never duplicated.



Offsets are committed automatically **before processing** the message. If the consumer crashes during processing, the message is skipped when restarted.

USE CASES

- Monitoring dashboards
- Metrics
- Analytics

Losing one event isn't catastrophic.

3. EXACTLY ONCE

"NO LOSS. NO DUPLICATES."

Guarantees that each message is produced and consumed exactly once across all broker failures and retries. Every message processed exactly one time.

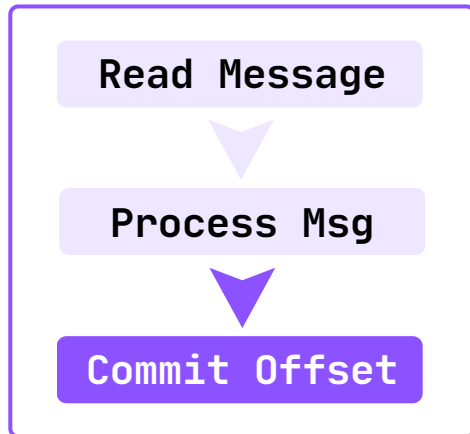
PROBLEMS :

MORE COMPLEXITY / MORE COORDINATION / MORE OVERHEAD/ LOWER THROUGHPUT

2. AT LEAST ONCE (ACKS=ALL)

"NEVER LOSE DATA. DUPLICATES ARE ACCEPTABLE."

Guarantees no messages are lost, but they may be delivered more than once in the event of failures or retries. If the broker fails to acknowledge, the producer retries.



Offsets are committed only after the message has been successfully processed.

If the consumer crashes mid-processing, it will re-read the uncommitted messages upon restart.

Requires **idempotent** handling on the consumer side. If not message is delivered twice.

Use Cases

- Order processing & Notifications
- Log processing
- Most enterprise systems

Because losing data is usually worse than duplication.

But distributed systems cannot magically know whether something succeeded. 2 Solutions ---> **Idempotent Producer / Transaction**

IDEMPOTENCY

Executing the same operation multiple times produces the same final result as executing it once.

IDEMPOTENT PRODUCER

Prevents duplicate records during producer retries using producer IDs and sequence numbers. It eliminates the "at-least-once" message duplication issue.

IDEMPOTENT CONSUMER

An idempotent consumer ensures that processing the same message multiple times produces the same result as processing it once. Tracks unique ID.

TRANSACTION

Ensure multiple Kafka operations either complete successfully together or are rolled back together, maintaining consistency.

Property	At Most Once	At Least Once	Exactly Once
Message Processing	0 or 1 time	1 or more times	Exactly 1 time
Data Loss	Possible	Not Possible	Not Possible
Duplicate Processing	Not Possible	Possible	Not Possible
Offset Commit	Before Processing	After Processing	Transactional Commit
Reliability	Low	High	Very High
Performance	High	Medium	Lower (additional overhead)
Complexity	Low	Medium	High
Failure Impact	Messages may be lost	Messages may be reprocessed	No loss, no duplicates
Typical Use Cases	Metrics, Monitoring, Logs	Orders, Payments, Notifications	Banking, Financial Transactions, Critical Systems
Primary Goal	Avoid Duplicates	Avoid Data Loss	Avoid Both

ACKS=ALL + MIN.INSYNC.REPLICAS + REPLICATION.FACTOR

`replication.factor` : Total copies maintained.

`min.insync.replicas` : Minimum replicas required to acknowledge a write.

e.g. `min.insync.replicas` = 2 → Leader + 1 ISR

`acks=all` : Producer waits for all required ISR replicas.

HOT PARTITION

Occurs when one partition receives disproportionately high traffic due to skewed partition keys.

PROBLEM : UNEVEN LOAD / INCREASED LAG / REDUCED THROUGHPUT

SOLUTION : BETTER PARTITION KEY / KEY SHARDING (SALTING)

KAFKA APIs

PRODUCER APIs

Write events to Kafka topics.

I/P : EVENT

TOPIC NAME

PARTITION (OPTIONAL)

KEY (OPTIONAL)

O/P : METADATA

TOPIC

PARTITION

OFFSET

CONSUMER APIs

Read events from Kafka topics.

I/P : TOPIC NAME

CONSUMER GROUP

OFFSET POSITION

O/P : STREAM OF EVENTS

ADMIN APIs

enables developers to manage and administer Kafka clusters programmatically. It provides a set of operations that can be used to create, delete, describe, and modify Kafka resources such as topics, brokers

I/P : CREATE TOPIC

DELETE TOPIC

INCREASE PARTITIONS

DESCRIBE CLUSTER

O/P : TOPIC DETAILS

BROKER DETAILS

PARTITION DETAILS

KAFKA CONNECT APIs

Move data between Kafka and external systems. Without this we need to write Custom Integration Code. MySQL/Postgres <---> Kafka

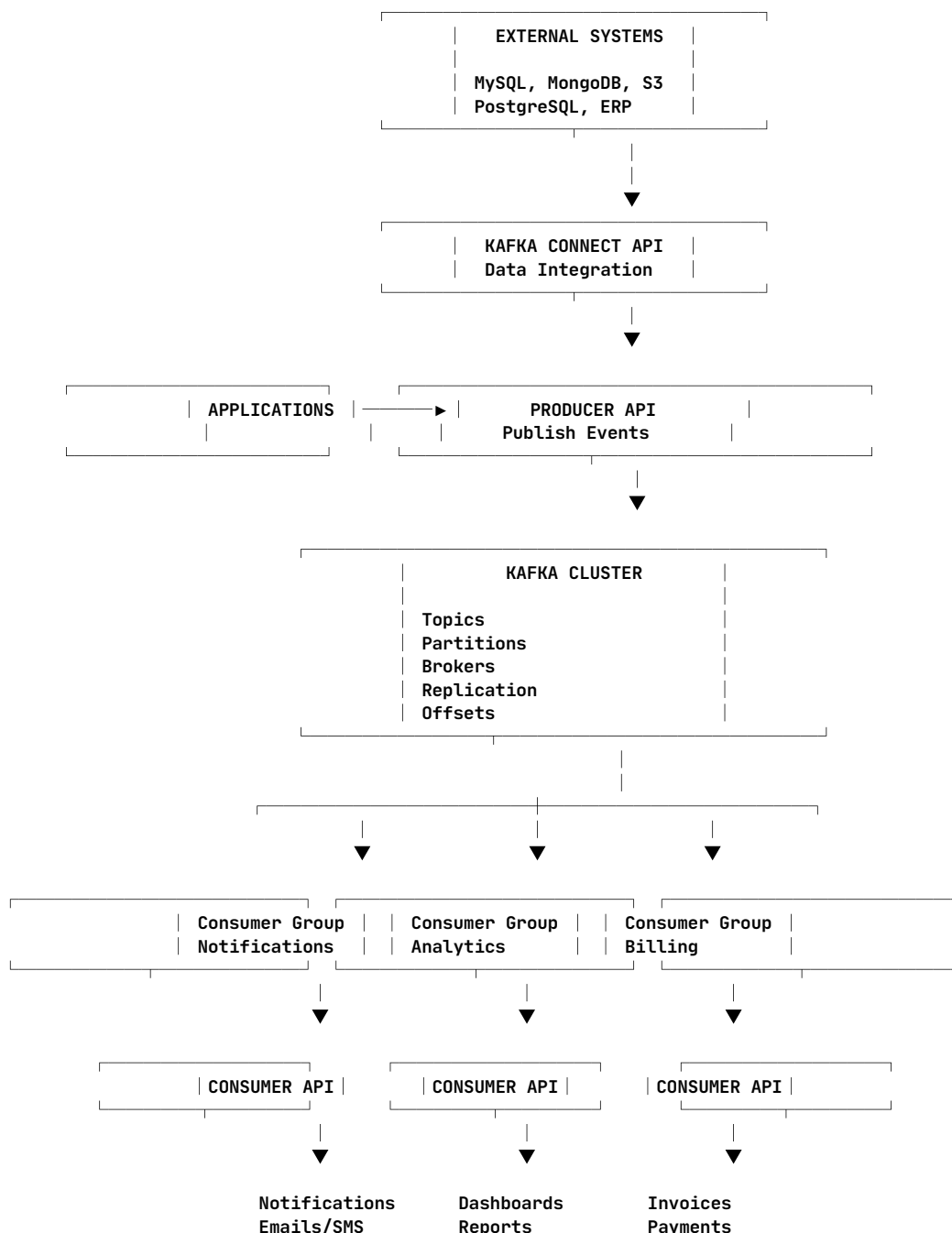
KAFKA APIs

KAFKA STREAMS API

Write events to Kafka topics. Process event streams in real time.
Kafka Streams is not for storing data. It is for Transforming Data
USED FOR : FILTER/AGREGATION/JOIN/WINDOWING

Why Not Consumer + Producer?

Kafka Streams is a higher-level library that does it more easily.



ZOOKEEPER AND KRaft

Early Kafka had a problem:

Broker 1 Broker 2 Broker 3

Who keeps track of:

- Which brokers are alive?
- Which broker is leader?
- Which partitions exist?
- Which topics exist?

Kafka couldn't manage this itself.

So Apache used:

Kafka + ZooKeeper

ZooKeeper is a distributed coordination service.

- Its job was not storing Kafka messages.
- Its job was storing Kafka metadata.
- Tracks active brokers.
- Elect new leader
- Every broker talks to ZooKeeper, and stored Topics, Partitions, Broker Information, Leader Information

Problems with ZooKeeper

- Additional Extra System
- More monitoring More servers.
- Scalability Issues
- Large Kafka clusters started hitting limitations.
- ZooKeeper became a bottleneck.

WHAT IS KRAFT

Kafka itself now manages:

- Topic Metadata
- Broker Metadata
- Leader Election
- Cluster Coordination

Raft is a distributed consensus protocol used to keep metadata consistent across controllers via Controller Nodes (Special Kafka nodes) Responsibilities:

- Metadata management
- Leader election
- Cluster coordination